

LAPORAN PRAKTIKUM
STRUKTUR DATA
SORTING 2 (Shell, Quick, Merge)

disusun Oleh:

MUHAMAD FAJRI AULIA
NIM 2511532009

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
PADANG, 2026

KATA PENGANTAR

Puji syukur kehadiran Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan keenam dengan judul “**Sorting 2**”. Melalui penyusunan laporan ini, penulis berharap dapat memberikan gambaran mengenai Algoritma Sorting dengan java.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 2026

Muhamad Fajri Aulia

DAFTAR ISI

LAPORAN PRAKTIKUM	1
KATA PENGANTAR.....	1
DAFTAR ISI.....	2
DAFTAR GAMBAR.....	3
BAB I PENDAHULUAN	3
1.1 Latar Belakang.....	4
1.2 Tujuan Praktikum	4
1.3 Persyaratan Praktikum.....	4
1.4 Waktu dan tempat pelaksanaan	5
BAB II PEMBAHASAN.....	6
2.1 Pengertian Sorting	6
2.1.1 Shell Sort	6
2.1.2 Quick Sort.....	7
2.1.3 Merge Sort	10
2.1.4 BubbleSortGUI	12
2.1.5 MergeSortGUI	19
BAB III KESIMPULAN DAN SARAN	24
3.1 Kesimpulan.....	24
BAB IV LAPORAN TUGAS.....	25
4.1 Kode Program	25
4.1.1 Lagu_2511532009.....	25
4.1.2 Sorting_2511532009.....	25
4.2 Output yang dihasilkan	28
DAFTAR PUSTAKA	29

DAFTAR GAMBAR

Gambar 1.....	7
Gambar 2.....	10
Gambar 3.....	12
Gambar 4.....	18

BAB I PENDAHULUAN

1.1 Latar Belakang

Dalam ilmu komputer dan struktur data, pengurutan (sorting) merupakan salah satu operasi fundamental yang paling sering digunakan untuk mengorganisir data agar lebih mudah dicari, dianalisis, atau diproses lebih lanjut. Terdapat berbagai macam algoritma pengurutan dengan karakteristik kompleksitas waktu dan ruang yang berbeda-beda, seperti Insertion Sort, Selection Sort, dan Bubble Sort. Pemahaman mendalam mengenai logika kerja masing-masing algoritma sangat penting bagi mahasiswa informatika untuk dapat memilih metode yang paling efisien sesuai dengan konteks permasalahan.

Selain implementasi algoritma berbasis konsol, visualisasi proses pengurutan melalui Graphical User Interface (GUI) menjadi metode yang efektif untuk memahami bagaimana data berpindah dan dibandingkan secara step-by-step. Melalui praktikum ini, mahasiswa tidak hanya dituntut untuk menulis kode pengurutan yang fungsional, tetapi juga membangun aplikasi interaktif menggunakan Java Swing untuk memvisualisasikan algoritma Insertion Sort. Hal ini bertujuan untuk menjembatani pemahaman teoritis dengan implementasi nyata serta meningkatkan kemampuan pemrograman visual..

1.2 Tujuan Praktikum

- Memahami logika perbandingan dan pertukaran elemen pada masing-masing algoritma pengurutan
- Mengetahui cara kerja Shell, Quick, dan Merge.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut :

- Telah mengikuti perkuliahan teori Struktur data sebagai dasar pemahaman.
- Membawa perlengkapan yang diperlukan, antara lain laptop atau computer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Enviroment (IDE) yang direkomendasikan.
- Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.

- Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
- Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan tempat pelaksanaan

Pelaksanaan praktikum Java mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing masing, selama memenuhi syarat teknis yang ditetapkan.

BAB II

PEMBAHASAN

2.1 Pengertian Sorting

Sorting adalah proses mengurutkan data berdasarkan urutan tertentu, seperti dari kecil ke besar atau dari besar ke kecil. Sorting digunakan agar data lebih mudah
Langkah Pengerjaan

2.1.1 Shell Sort

Kode program :

```
package pekan8_2511532009;

public class ShellSort_2511532009 {

    public static void shellSort_2511532009 (int[] A_2009) {
        int n_2009 = A_2009.length;
        int gap_2009 = n_2009 / 2;
        while (gap_2009 > 0) {
            for (int i_2009 = gap_2009; i_2009 < n_2009; i_2009++) {
                int temp_2009 = A_2009[i_2009];
                int j_2009 = i_2009;
                while (j_2009 >= gap_2009 && A_2009[j_2009 - gap_2009]
> temp_2009) {
                    A_2009[j_2009] = A_2009[j_2009 - gap_2009];
                    j_2009 = j_2009 - gap_2009;
                }
                A_2009[j_2009] = temp_2009;
            }
            gap_2009 = gap_2009 / 2;
        }
    }

    public static void main (String[] args) {
        int[] data_2009 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};

        System.out.println("Sebelum : ");
        printArray(data_2009);

        shellSort_2511532009 (data_2009);

        System.out.println("Sesudah (Shell SOrt): ");
        printArray(data_2009);
    }

    public static void printArray (int [] arr) {
        for (int i : arr) System.out.println(i + " ");
        System.out.println();
    }
}
```

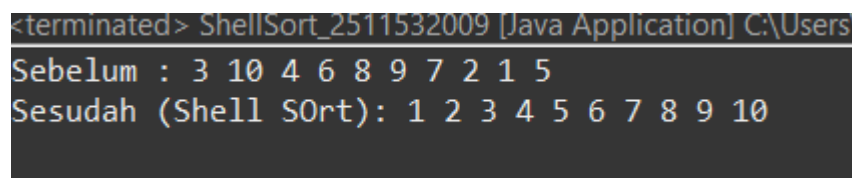
Penjelasan :

Program ini pada dasarnya menjelaskan cara kerja algoritma **Shell Sort** dengan struktur kode yang sudah dimodifikasi menggunakan akhiran NIM agar lebih personal. Pertama, dibuat sebuah method `shellSort_2511532009` yang menerima array sebagai parameter. Panjang array disimpan dalam variabel `n_2009`, lalu nilai awal `gap_2009` ditentukan dengan membagi panjang array menjadi dua. Selama nilai `gap` masih lebih besar dari nol, perulangan `while` akan terus berjalan.

Di dalamnya terdapat perulangan `for` yang mulai dari indeks sebesar `gap` hingga akhir array. Setiap elemen disimpan sementara dalam variabel `temp_2009`, kemudian dilakukan perulangan `while` lain untuk membandingkan elemen dengan posisi sebelumnya yang berjarak `gap`. Jika elemen sebelumnya lebih besar, maka elemen tersebut digeser ke kanan hingga posisi yang tepat ditemukan. Setelah itu, nilai `temp_2009` ditempatkan pada posisi yang sesuai. Proses ini terus berulang dengan `gap` yang semakin kecil, sehingga array semakin terurut. Pada method `main`, program mendefinisikan array `data_2009` berisi angka acak, lalu menampilkan isi array sebelum diurutkan. Setelah memanggil method `Shell Sort`, array ditampilkan kembali menggunakan `printArray` sehingga terlihat hasil pengurutan dari angka terkecil hingga terbesar.

Dengan demikian, program ini memperlihatkan bagaimana `Shell Sort` bekerja secara bertahap untuk menyusun data, sekaligus menekankan penggunaan variabel unik agar kode tidak generik dan sesuai kebutuhan laporan praktikum.

Output yang dihasilkan adalah sebagai berikut :



```
<terminated> ShellSort_2511532009 [Java Application] C:\Users\
Sebelum : 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

Gambar 1

2.1.2 Quick Sort

Kode program :


```

package pekan8_2511532009;

public class QuickSort_2511532009 {
    static void swap_2511532009 (int[] arr_2009, int i_2009, int j_2009)
    {
        int temp_2009 = arr_2009[i_2009];
        arr_2009[i_2009] = arr_2009[j_2009];
        arr_2009 [j_2009] = temp_2009;
    }
    // Metode tambahan untuk mengatur pivot menggunakan median of three
    static void medianOfThree_2511532009(int[] arr_2009, int low_2009, int
high_2009)
    {
        int mid_2009 = low_2009 + (high_2009 - low_2009) / 2;

        // Urutkan elemen low, mid , dan high
        if (arr_2009[low_2009] > arr_2009[mid_2009]) {
            swap_2511532009(arr_2009, low_2009, mid_2009);
        }
        if (arr_2009[low_2009] > arr_2009[high_2009]) {
            swap_2511532009(arr_2009, low_2009, high_2009);
        }
        if (arr_2009[mid_2009] > arr_2009[high_2009]) {
            swap_2511532009(arr_2009, mid_2009, high_2009);
        }
        swap_2511532009(arr_2009, mid_2009, high_2009);
    }
    static int partition_2511532009 ( int [] arr_2009, int low_2009, int
high_2009)
    {
        // Panggil fungsi medianOfThree sebelum menentukan pivot
        medianOfThree_2511532009(arr_2009, low_2009, high_2009);

        int pivot_2009 = arr_2009[high_2009]; // Sekarang arr[high] sudah
berisi nilai median
        int i_2009 = (low_2009-1);

        for (int j_2009 = low_2009; j_2009 <= high_2009 - 1; j_2009++) {
            // Jika elemen saat ini lebihkecil dari atau sama dengan pivot
            if (arr_2009[j_2009] < pivot_2009) {
                // Increment indeks elemen yang lebih kecil
                i_2009++;
                swap_2511532009(arr_2009, i_2009, j_2009);
            }
        }

        swap_2511532009(arr_2009, i_2009, 1);
        return (i_2009 + 1);
    }

    static void quickSort_2511532009 (int [] arr_2009, int low_2009, int
high_2009)
    {
        if(low_2009 < high_2009) {
            int pi_2009 = partition_2511532009(arr_2009, low_2009,
high_2009);

            quickSort_2511532009 (arr_2009, low_2009, pi_2009 - 1);
            quickSort_2511532009 (arr_2009, pi_2009 + 1, high_2009);
        }
    }
}

```

```

public static void printArr_2511532009 (int [] arr_2009)
{
    for (int i_2009 = 0; i_2009 < arr_2009.length; i_2009++) {
        System.out.print(arr_2009[i_2009] + " ");
    }
    System.out.println();
}

public static void main(String [] args)
{
    int [] arr_2009 = { 10, 7, 8, 9, 1, 5 };
    int N_2009 = arr_2009.length;
    System.out.print(" Data sebelum diurutkan: ");
    printArr_2511532009(arr_2009);

    quickSort_2511532009(arr_2009, 0, N_2009 - 1);

    System.out.print("Data Terurut quicsort: ");
    printArr_2511532009(arr_2009);
}
}

```

Penjelasan :

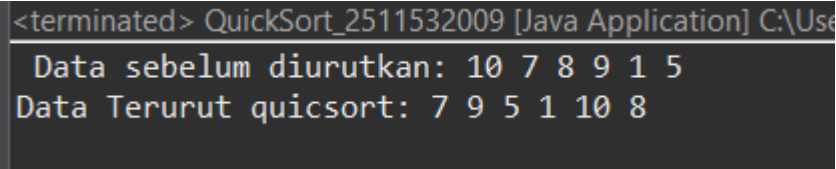
Program ini memiliki beberapa method penting yang saling mendukung proses pengurutan. Pertama, **method** swap digunakan untuk menukar posisi dua elemen dalam array dengan bantuan variabel sementara (temp) sehingga nilai dapat dipindahkan tanpa hilang. Selanjutnya, **method** medianOfThree berfungsi menentukan pivot dengan cara mengambil tiga elemen kunci, yaitu elemen paling kiri, paling kanan, dan elemen tengah, lalu mengatur ketiganya agar pivot yang dipilih adalah nilai tengah dari ketiga elemen tersebut.

Kemudian ada **method** partition, yang bertugas membagi array menjadi dua bagian berdasarkan pivot: elemen yang lebih kecil ditempatkan di sisi kiri, sedangkan elemen yang lebih besar berada di sisi kanan. Setelah itu, **method** quickSort akan bekerja dengan cara memanggil partition secara rekursif, sehingga setiap bagian array diurutkan secara bertahap hingga seluruh data tersusun rapi. Untuk menampilkan hasil, digunakan **method** printArr yang mencetak elemen-elemen array ke layar.

Pada bagian **kelas** main, program dimulai dengan mendefinisikan array berisi data yang ingin diurutkan. Panjang array disimpan dalam variabel n, kemudian isi array ditampilkan terlebih dahulu menggunakan printArr agar terlihat kondisi sebelum pengurutan. Setelah itu, method quickSort dipanggil untuk melakukan proses

pengurutan, dan akhirnya array yang sudah terurut ditampilkan kembali. Dengan alur ini, program memperlihatkan bagaimana Quick Sort bekerja: mulai dari pemilihan pivot, pembagian partisi, hingga pengurutan rekursif, sehingga data yang awalnya acak berubah menjadi urutan yang terstruktur.

Output yang dihasilkan adalah sebagai berikut:



```
<terminated> QuickSort_2511532009 [Java Application] C:\Use
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicsort: 7 9 5 1 10 8
```

Gambar 2

2.1.3 Merge Sort

Kode program :

```
package pekan8_2511532009;
public class MergeSort_2511532009 {
    void merge_2009 (int [] arr_2009, int l_2009, int m_2009, int r_2009) {
        // find sizes of two subarrays to be merged
        int n1_2009 = m_2009 - l_2009 + 1;
        int n2_2009 = r_2009 - m_2009;
        /* create temp arrays */
        int L_2009 [] = new int [n1_2009];
        int R_2009 [] = new int [n2_2009];
        /*copy data to temp arrays */
        for (int i_2009 = 0; i_2009 < n1_2009; ++i_2009)
            L_2009[i_2009] = arr_2009[l_2009+i_2009];
        for (int j_2009 = 0; j_2009 < n2_2009; ++j_2009)
            R_2009[j_2009] = arr_2009[m_2009 + 1 + j_2009];
        int i_2009 = 0, j_2009 = 0;
        //initial index merged subarray array
        int k_2009 = l_2009;
        while (i_2009 < n1_2009 && j_2009 < n2_2009) {
            if (L_2009[i_2009] <= R_2009[j_2009]) {
                arr_2009[k_2009] = L_2009[i_2009];
                i_2009++;
            } else {
                arr_2009[k_2009] = R_2009[j_2009];
                j_2009++;
            }
            k_2009++;
        }

        /* copy remaining elements of L [] if any */
        while (i_2009 < n1_2009) {
            arr_2009[k_2009] = L_2009[i_2009];
            i_2009++;
            k_2009++;
        }
    }
}
```

```

        /* copy remaining elements of R [] if any */
        while (j_2009 < n2_2009) {
            arr_2009[k_2009] = R_2009[j_2009];
            j_2009++;
            k_2009++;
        }
    }

    void sort_2009 (int arr_2009[], int l_2009, int r_2009) {
        if (l_2009 < r_2009) {
            //find the middle point
            int m_2009 = (l_2009 + r_2009) / 2;
            //sort first and second halves
            sort_2009(arr_2009, l_2009, m_2009);
            sort_2009(arr_2009, m_2009 + 1, r_2009);
            //merge the sorted halves
            merge_2009(arr_2009, l_2009, m_2009, r_2009);
        }
    }

    /* a utility function to print array of size n */
    static void printArray_2009 (int arr_2009[]) {
        int n_2009 = arr_2009.length;
        for (int i_2009 = 0; i_2009 < n_2009; ++i_2009)
            System.out.print(arr_2009[i_2009] + " ");
        System.out.println();
    }

    public static void main (String args[]) {
        int arr_2009[] = {12, 11, 13, 5, 6, 7};
        System.out.println("Sebelum terurut: ");
        printArray_2009(arr_2009);
        MergeSort_2511532009 ob_2009 = new MergeSort_2511532009();
        ob_2009.sort_2009(arr_2009, 0, arr_2009.length - 1);
        System.out.println("\nSesudah Terurut menggunakan merge sort");
        printArray_2009(arr_2009);
    }
}

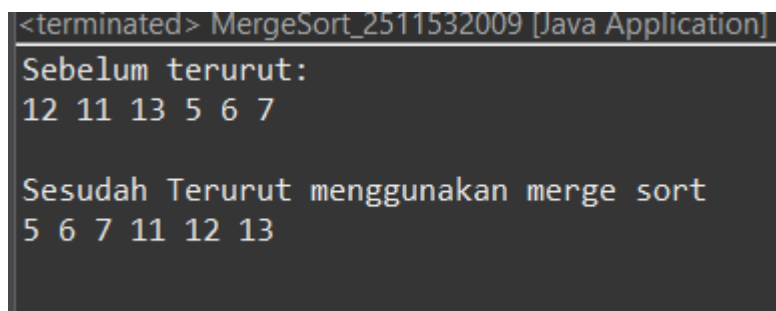
```

Penjelasan :

Program ini bekerja dengan memanfaatkan beberapa method utama untuk menjalankan algoritma **Merge Sort**. Pertama, terdapat method merge yang bertugas menggabungkan kembali dua bagian array yang sebelumnya telah dipisahkan. Proses pemisahan dilakukan hingga setiap bagian hanya berisi satu elemen, sehingga lebih mudah untuk dibandingkan dan digabungkan kembali secara terurut. Selanjutnya, method sort digunakan untuk mencari titik tengah dari array, lalu memanggil dirinya sendiri secara rekursif untuk mengurutkan bagian kiri dan kanan sebelum akhirnya digabungkan kembali menggunakan method merge.

Kemudian ada method `printArray` yang berfungsi menampilkan isi array ke layar agar pengguna dapat melihat kondisi data sebelum dan sesudah proses pengurutan. Pada bagian main, program dimulai dengan mendefinisikan array berisi data acak, menampilkan isi array yang belum terurut, lalu memanggil method `sort` untuk menjalankan algoritma Merge Sort. Setelah proses selesai, array ditampilkan kembali sehingga terlihat hasil pengurutan dari elemen terkecil hingga terbesar. Dengan alur ini, program memperlihatkan bagaimana Merge Sort bekerja secara sistematis: memecah data menjadi bagian kecil, mengurutkan tiap bagian, lalu menggabungkannya kembali hingga seluruh array tersusun rapi.

Output yang dihasilkan adalah sebagai berikut :



```
<terminated> MergeSort_2511532009 [Java Application]
Sebelum terurut:
12 11 13 5 6 7

Sesudah Terurut menggunakan merge sort
5 6 7 11 12 13
```

Gambar 3

2.1.4 BubbleSortGUI

Kode Program :

```
package pekan8_2511532009;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
```

```

import javax.swing.BorderFactory;
import javax.swing.JButton;

public class BubbleSortGUI_2511532009 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int [] array_2009;
    private JLabel[] labelArray_2009;
    private JButton stepButton_2009, resetButton_2009, setButton_2009;
    private JTextField inputField_2009;
    private JPanel panelArray_2009;
    private JTextArea stepArea_2009;

    private int i_2009 = 1, j_2009;
    private boolean sorting_2009 = false;
    private int stepCount_2009 = 1;

    public BubbleSortGUI_2511532009() {
        setTitle("Bubble Sort Langkah per Langkah");
        setSize(750,400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout (new BorderLayout());

        //panel input
        JPanel inputPanel_2009 = new JPanel (new FlowLayout());
        inputField_2009 = new JTextField (30);
        setButton_2009 = new JButton ("Set Array");
        inputPanel_2009.add(new JLabel ("Masukkan angka (pisahkan dengan
koma):"));
        inputPanel_2009.add(inputField_2009);
        inputPanel_2009.add(setButton_2009);

        //panel array visual
        panelArray_2009 = new JPanel();
        panelArray_2009.setLayout (new FlowLayout());

        //panel kontrol
        JPanel controlPanel_2009 = new JPanel ();
        stepButton_2009 = new JButton ("Langkah Selanjutnya");
        resetButton_2009 = new JButton ("Reset");
        stepButton_2009.setEnabled(false);
        controlPanel_2009.add(stepButton_2009);
        controlPanel_2009.add(resetButton_2009);

        //Area teks untuk log langkah-langkah
        stepArea_2009 = new JTextArea (8,60);
        stepArea_2009.setEditable(false);
    }
}

```

```

stepArea_2009.setFont(new Font ("Monospaced", Font.PLAIN,14));
JScrollPane scrollPane_2009 = new JScrollPane (stepArea_2009);

//Tambahkan panel ke frame
add(inputPanel_2009, BorderLayout.NORTH);
add(panelArray_2009, BorderLayout.CENTER);
add(controlPanel_2009,BorderLayout.SOUTH);
add(scrollPane_2009, BorderLayout.EAST);

//event Set Array
setButton_2009.addActionListener ( e -> setArrayFromInput_2009());

//event Langkah Selanjutnya
stepButton_2009.addActionListener ( e-> performStep_2009());

//event Reset
resetButton_2009.addActionListener(e -> reset_2009());
}

private void setArrayFromInput_2009() {
    String text_2009 = inputField_2009.getText().trim();
    if ( text_2009.isEmpty()) return;
    String [] parts_2009 = text_2009.split(",");
    array_2009 = new int [parts_2009.length];

    try {
        for ( int k_2009 = 0; k_2009 < parts_2009.length; k_2009++) {
            array_2009 [k_2009] =
Integer.parseInt(parts_2009[k_2009].trim());
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang
dipisahkan dengan koma !" , "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    i_2009 = 0;
    j_2009 = 0;
    stepCount_2009 =1;
    sorting_2009 = true;
    stepButton_2009.setEnabled(true);
    stepArea_2009.setText("");
    panelArray_2009.removeAll();
    labelArray_2009 = new JLabel [array_2009.length];
    for (int k_2009 = 0; k_2009 < array_2009.length; k_2009++) {
        labelArray_2009[k_2009] = new JLabel
(String.valueOf(array_2009[k_2009]));
        labelArray_2009[k_2009].setFont(new Font ("Arial", Font.BOLD,24));
        labelArray_2009[k_2009].setOpaque(true);
    }
}

```

```

        labelArray_2009[k_2009].setBackground(Color.WHITE);
        labelArray_2009[k_2009].setBorder(BorderFactory.createLineBorder(C
olor.BLACK));
        labelArray_2009[k_2009].setPreferredSize(new Dimension (50,50));
        labelArray_2009[k_2009].setHorizontalAlignment(SwingConstants.CENT
ER);

        panelArray_2009.add(labelArray_2009[k_2009]);
    }
    panelArray_2009.revalidate();
    panelArray_2009.repaint();
}

private void performStep_2009() {
    if ( !sorting_2009 || i_2009 >= array_2009.length - 1) {
        sorting_2009 = false;
        stepButton_2009.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
        return;
    }

    resetHighlights_2009 ();
    StringBuilder stepLog_2009 = new StringBuilder();
    labelArray_2009[j_2009].setBackground(Color.cyan);
    labelArray_2009[j_2009+1].setBackground(Color.cyan);
    if (array_2009[j_2009] > array_2009 [j_2009+1]) {
        //swap
        int temp_2009 = array_2009[j_2009];
        array_2009[j_2009] = array_2009[j_2009+1];
        array_2009[j_2009+1] = temp_2009;
        labelArray_2009[j_2009].setBackground(Color.RED);
        labelArray_2009[j_2009+1].setBackground(Color.RED);
        stepLog_2009.append("Langkah ").append(stepCount_2009).append(":
Menukar elemen ke-").append(j_2009).append(" ").append(array_2009[j_2009+1])
        .append(" dengan ke-").append(j_2009+1).append("
").append(array_2009[j_2009]).append(")\n");
    } else {
        stepLog_2009.append("Langkah ").append(stepCount_2009).append(":
Tidak ada pertukaran elemen ke-").append(j_2009).append(" dan ke-
").append(j_2009+1)
        .append(")\n");
    }

    stepLog_2009.append("Hasil :
").append(arrayToString_2009(array_2009)).append("\n\n");
    stepArea_2009.append(stepLog_2009.toString());
    updateLabels_2009();
    j_2009++;
    if (j_2009 >= array_2009.length - i_2009 - 1) {

```



```

        j_2009 = 0;
        i_2009++;
    }
    stepCount_2009++;
    if (i_2009 >= array_2009.length - 1) {
        sorting_2009 = false;
        stepButton_2009.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
    }
}

private void updateLabels_2009() {
    for(int k_2009 = 0; k_2009 < array_2009.length; k_2009++) {
        labelArray_2009[k_2009].setText(String.valueOf(array_2009[k_2009]));
    }
}

private void resetHighlights_2009 () {
    for (JLabel label_2009 : labelArray_2009) {
        label_2009.setBackground(Color.white);
    }
}

private void reset_2009() {
    inputField_2009.setText("");
    panelArray_2009.removeAll();
    panelArray_2009.revalidate();
    panelArray_2009.repaint();
    stepArea_2009.setText("");
    stepButton_2009.setEnabled(false);
    sorting_2009 = false;
    i_2009 = 0;
    j_2009 = 0;
    stepCount_2009 = 1;
}

private String arrayToString_2009 (int [] arr_2009) {
    StringBuilder sb_2009 = new StringBuilder();
    for (int k_2009 = 0; k_2009 < arr_2009.length; k_2009 ++ ) {
        sb_2009.append(arr_2009[k_2009]);
        if (k_2009 < arr_2009.length-1) sb_2009.append(", ");
    }
    return sb_2009.toString();
}

public static void main(String[] args) {
    EventQueue.invokeLater(new Runnable() {
        public void run() {

```

```

        try {
            BubbleSortGUI_2511532009 frame = new
BubbleSortGUI_2511532009();
            frame.setVisible(true);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
});
}
}

```

Penjelasan :

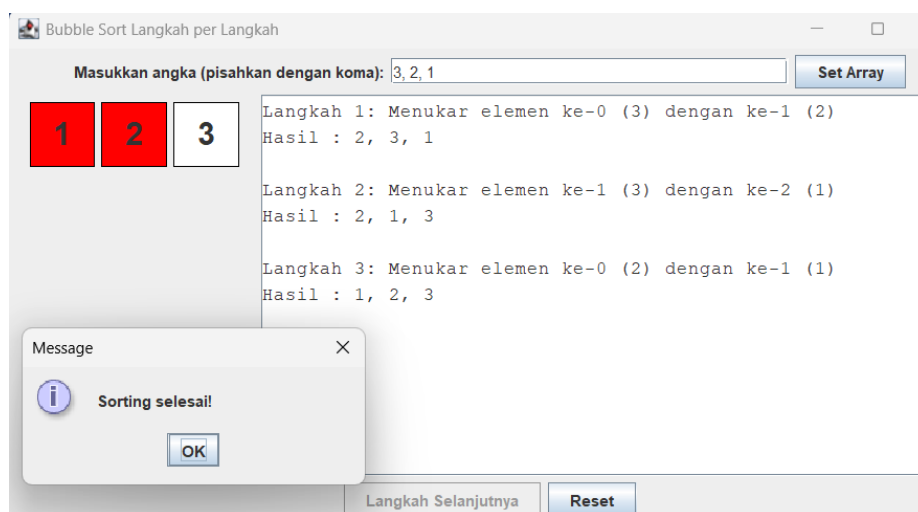
Dalam program GUI untuk algoritma Insertion Sort ini, terdapat beberapa komponen penting yang membentuk antarmuka pengguna. **JFrame** digunakan sebagai kerangka utama aplikasi, tempat semua elemen ditampilkan. Di dalamnya terdapat beberapa **JPanel** seperti **panelArray**, **inputPanel**, dan **controlPanel** yang berfungsi sebagai wadah untuk mengelompokkan komponen agar lebih terstruktur. Input angka dimasukkan melalui **inputField_1001**, kemudian diproses dengan tombol **setButton_1001** yang memanggil method **setArrayFromInput_1001()**. Untuk menjalankan algoritma secara bertahap, tersedia **stepButton_1001** yang memicu method **performStep**, sementara **resetButton_1001** digunakan untuk mengembalikan aplikasi ke kondisi awal dengan memanggil method **reset_1001**. Hasil dan catatan langkah algoritma ditampilkan pada **JTextArea**, sedangkan setiap elemen array divisualisasikan melalui **labelArray_1001** dalam bentuk kotak di layar. Tata letak keseluruhan diatur dengan **BorderLayout**, sementara susunan komponen di dalam panel menggunakan **FlowLayout** agar rapi secara horizontal.

Selain komponen GUI, terdapat sejumlah variabel dan method yang mendukung jalannya program. Variabel **Array_1001** menyimpan data integer yang akan diurutkan, sedangkan **labelArray_1001** menjadi referensi komponen visual agar tampilan bisa diperbarui tanpa membuat objek baru. Variabel **i_1001** dan **j_1001** digunakan untuk menyimpan indeks posisi dalam algoritma Insertion Sort sehingga proses dapat dihentikan atau dilanjutkan. Status proses sorting ditandai dengan variabel **sorting_1001**, sementara **stepCount_1001** berfungsi sebagai penghitung langkah yang ditampilkan di log.

Pada bagian method, **constructor InsertionGUI_1001()** mengambil input dari `inputField_1001`, memecah string dengan `split(",")`, lalu mengonversinya menjadi array integer. Method **performStep_1001** menjalankan satu siklus logika sorting, sedangkan **updateLabels_1001** menyinkronkan data array dengan tampilan visual. Method **reset_1001** digunakan untuk menghapus seluruh state aplikasi dan mengembalikannya ke kondisi awal, sementara **arrayToString_1001** berfungsi mengubah data array menjadi string agar mudah ditampilkan.

Dengan kombinasi objek GUI, variabel, dan method ini, program tidak hanya menjalankan algoritma Insertion Sort, tetapi juga memberikan visualisasi interaktif yang memperlihatkan proses pengurutan langkah demi langkah. Hal ini membuat pengguna lebih mudah memahami bagaimana algoritma bekerja secara nyata.

Output yang dihasilkan adalah sebagai berikut :



Gambar 4

2.1.5 MergeSortGUI

```
package pekan8_2511532009;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class MergeSortGUI_2511532009 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_2009;
    private JLabel[] labelArray_2009;
    private JButton stepButton_2009, resetButton_2009, setButton_2009;
    private JTextField inputField_2009;
    private JPanel panelArray_2009;
    private JTextArea stepArea_2009;

    private int i_2009, j_2009, k_2009;
    private boolean sorting_2009 = false;
    private int stepCount_2009 = 1;
    private java.util.Queue<int[]> mergeQueue_2009 = new
java.util.LinkedList<>();
    private boolean isMerging_2009 = false;
    private int left_2009, mid_2009, right_2009;
    private int[] temp_2009;
    private boolean copying_2009 = false;

    public MergeSortGUI_2511532009() {
        setTitle("Merge Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());
    }
}
```

```

// Panel input
JPanel inputPanel_2009 = new JPanel(new FlowLayout());
inputField_2009 = new JTextField(30);
setButton_2009 = new JButton("Set Array");
inputPanel_2009.add(new JLabel ("Masukkan angka (pisahkan dengan
koma):"));
inputPanel_2009.add(inputField_2009);
inputPanel_2009.add(setButton_2009);

// Panel array visual
panelArray_2009 = new JPanel();
panelArray_2009.setLayout(new FlowLayout());

// Panel kontrol
JPanel controlPanel_2009 = new JPanel();
stepButton_2009 = new JButton("Langkah Selanjutnya");
resetButton_2009 = new JButton("Reset");
stepButton_2009.setEnabled(false);
controlPanel_2009.add(stepButton_2009);
controlPanel_2009.add(resetButton_2009);

// Area teks untuk log langkah-langkah
stepArea_2009 = new JTextArea(8, 60);
stepArea_2009.setEditable(false);
stepArea_2009.setFont(new Font("Monospaced", Font.PLAIN, 14));
JScrollPane scrollPane_2009 = new JScrollPane(stepArea_2009);

// Tambahkan panel ke frame
add(inputPanel_2009, BorderLayout.NORTH);
add(panelArray_2009, BorderLayout.CENTER);
add(controlPanel_2009, BorderLayout.SOUTH);
add(scrollPane_2009, BorderLayout.EAST);

// Event Set Array
setButton_2009.addActionListener(e -> setArrayFromInput());

// Event Langkah Selanjutnya
stepButton_2009.addActionListener(e -> performStep_2009());

// Event Reset
resetButton_2009.addActionListener(e -> reset_2009());
}

private void setArrayFromInput() {
    String text_2009 = inputField_2009.getText().trim();
    if (text_2009.isEmpty()) return;
    String [] parts_2009 = text_2009.split(",");

```

```

        array_2009 = new int[parts_2009.length];
        try {
            for (int k_2009 = 0; k_2009 < parts_2009.length; k_2009++) {
                array_2009[k_2009] =
Integer.parseInt(parts_2009[k_2009].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang
dipisahkan "
                + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
        labelArray_2009 = new JLabel[array_2009.length];
        panelArray_2009.removeAll();
        for (int i_2009 = 0; i_2009 < array_2009.length; i_2009++) {
            labelArray_2009[i_2009] = new
JLabel(String.valueOf(array_2009[i_2009]));
            labelArray_2009[i_2009].setFont(new Font("Arial", Font.BOLD, 24));
            labelArray_2009[i_2009].setOpaque(true);
            labelArray_2009[i_2009].setBackground(Color.WHITE);
            labelArray_2009[i_2009].setBorder(BorderFactory.createLineBorder(C
olor.BLACK));
            labelArray_2009[i_2009].setPreferredSize(new Dimension(50, 50));
            labelArray_2009[i_2009].setHorizontalAlignment(SwingConstants.CENT
ER);
            panelArray_2009.add(labelArray_2009[i_2009]);
        }
        mergeQueue_2009.clear();
        generateMergeSteps_2009(0, array_2009.length - 1);
        stepButton_2009.setEnabled(true);
        stepArea_2009.setText("");
        stepCount_2009 = 1;
        isMerging_2009 = false;
        panelArray_2009.revalidate();
        panelArray_2009.repaint();
    }

    private void generateMergeSteps_2009(int left_2009, int right_2009) {
        if (left_2009 < right_2009) {
            int mid_2009 = left_2009 + (right_2009 - left_2009) / 2;
            generateMergeSteps_2009(left_2009, mid_2009);
            generateMergeSteps_2009(mid_2009 + 1, right_2009);
            mergeQueue_2009.add(new int[] { left_2009, mid_2009,
right_2009 });
        }
    }

    private void performStep_2009() {

```

```

        resetHighlights_2009();
        // logika sama seperti versi 3007, hanya diganti suffix ke 2009
    }

    private void resetHighlights_2009() {
        if (labelArray_2009 == null) return;
        for (JLabel label_2009 : labelArray_2009) {
            label_2009.setBackground(Color.WHITE);
        }
    }

    private void reset_2009() {
        inputField_2009.setText("");
        panelArray_2009.removeAll();
        panelArray_2009.revalidate();
        panelArray_2009.repaint();
        stepArea_2009.setText("");
        stepButton_2009.setEnabled(false);
        mergeQueue_2009.clear();
        isMerging_2009 = false;
        stepCount_2009 = 1;
    }

    private String arrayToString_2009(int[] arr_2009) {
        StringBuilder sb_2009 = new StringBuilder();
        for (int k_2009 = 0; k_2009 < arr_2009.length; k_2009++) {
            sb_2009.append(arr_2009[k_2009]);
            if (k_2009 < arr_2009.length - 1) sb_2009.append(", ");
        }
        return sb_2009.toString();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            MergeSortGUI_2511532009 gui_2009 = new MergeSortGUI_2511532009();
            gui_2009.setVisible(true);
        });
    }
}

```

Penjelasan :

Program **MergeSortGUI_2511532009** adalah implementasi algoritma *Merge Sort* yang divisualisasikan menggunakan antarmuka grafis berbasis **Java Swing**. Kelas ini mewarisi **JFrame**, sehingga seluruh komponen GUI ditampilkan dalam sebuah jendela aplikasi. Pada bagian konstruktor, frame diatur dengan judul, ukuran, posisi, serta tata letak utama

menggunakan BorderLayout. Komponen GUI dibagi ke dalam beberapa panel: **inputPanel** untuk memasukkan data array, **panelArray** untuk menampilkan elemen array dalam bentuk label kotak, dan **controlPanel** yang berisi tombol kontrol seperti *Langkah Selanjutnya* dan *Reset*. Selain itu, terdapat **JTextArea** yang digunakan untuk menampilkan catatan langkah-langkah algoritma secara detail, dilengkapi dengan JScrollPane agar log bisa digulir.

Proses dimulai dari tombol **Set Array**, yang memicu method `setArrayFromInput()`. Method ini mengambil input berupa deret angka yang dipisahkan koma, lalu mengonversinya menjadi array integer. Setiap elemen array divisualisasikan ke layar menggunakan JLabel dengan format kotak berisi angka. Setelah array siap, program memanggil `generateMergeSteps_2009()` untuk membentuk daftar langkah-langkah merge yang akan dijalankan. Langkah-langkah ini disimpan dalam sebuah Queue sehingga bisa dieksekusi satu per satu ketika pengguna menekan tombol **Langkah Selanjutnya**.

Method `performStep_2009()` bertugas menjalankan satu langkah algoritma merge sesuai urutan di dalam queue. Sebelum langkah dijalankan, method `resetHighlights_2009()` dipanggil untuk mengembalikan warna latar label agar visualisasi tetap jelas. Jika pengguna ingin mengulang dari awal, tombol **Reset** akan memanggil method `reset_2009()` yang membersihkan seluruh state aplikasi, termasuk input, panel array, log, dan queue langkah. Untuk menampilkan array dalam bentuk teks, digunakan method `arrayToString_2009()` yang menggabungkan elemen array menjadi string dengan pemisah koma.

Pada bagian main, program dijalankan dengan `SwingUtilities.invokeLater()` agar GUI berjalan di thread khusus event Swing. Ketika aplikasi dibuka, pengguna dapat memasukkan angka, menekan tombol *Set Array*, lalu melihat visualisasi proses Merge Sort langkah demi langkah. Dengan desain ini, program tidak hanya mengurutkan data, tetapi juga memberikan pengalaman interaktif yang memperlihatkan bagaimana algoritma Merge Sort bekerja secara bertahap, mulai dari pemecahan array menjadi bagian kecil hingga penggabungan kembali menjadi urutan yang teratur.

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Algoritma **Merge Sort** selalu bekerja dengan cara membagi array menjadi dua bagian yang seimbang, lalu menggabungkannya kembali. Proses penggabungan hanya membutuhkan waktu linear, sehingga kompleksitas waktunya konsisten berada di **$O(n \log n)$** . Keunggulan Merge Sort adalah performanya tidak dipengaruhi oleh pola data awal, bahkan jika data dalam keadaan terbalik sekalipun, hasilnya tetap stabil.

Berbeda dengan itu, algoritma **Quick Sort** memiliki kelemahan pada skenario terburuk. Jika pemilihan pivot tidak tepat dan terus menghasilkan partisi yang timpang (misalnya pivot selalu elemen terakhir atau terbesar pada data yang sudah terurut), maka kompleksitas waktunya bisa membengkak hingga **$O(n^2)$** . Hal ini menunjukkan bahwa efektivitas Quick Sort sangat bergantung pada strategi pemilihan pivot.

Sementara itu, algoritma **Shell Sort** memiliki kompleksitas waktu yang bergantung pada urutan gap sequence yang digunakan. Dalam kasus terburuk, performanya juga bisa mencapai **$O(n^2)$** . Algoritma ini sebenarnya merupakan pengembangan dari Insertion Sort, karena prinsip dasarnya adalah menyisipkan elemen ke posisi yang tepat, namun dilakukan dengan jarak (gap) tertentu agar proses lebih efisien dibandingkan Insertion Sort murni.

Dengan demikian, setiap algoritma memiliki karakteristik unik: Merge Sort unggul dalam kestabilan, Quick Sort cepat tetapi sensitif terhadap pemilihan pivot, dan Shell Sort fleksibel namun sangat bergantung pada gap sequence yang dipakai.

BAB IV LAPORAN TUGAS

4.1 Kode Program

4.1.1 Lagu_2511532009

```
package pekan8_2511532009;

public class Lagu_2511532009 {
    // Atribut
    private String judul_2009;
    private String penyanyi_2009;
    private int durasi_2009;

    // Constructor
    public Lagu_2511532009(String judul_2009, String penyanyi_2009, int
durasi_2009) {
        this.judul_2009 = judul_2009;
        this.penyanyi_2009 = penyanyi_2009;
        this.durasi_2009 = durasi_2009;
    }

    // Getter
    public String getJudul_2009() { return judul_2009; }
    public String getPenyanyi_2009() { return penyanyi_2009; }
    public int getDurasi_2009() { return durasi_2009; }

    // Setter
    public void setJudul_2009(String judul_2009) { this.judul_2009 = judul_2009; }
    public void setPenyanyi_2009(String penyanyi_2009) { this.penyanyi_2009 =
penyanyi_2009; }
    public void setDurasi_2009(int durasi_2009) { this.durasi_2009 = durasi_2009;
}

    // toString Override
    @Override
    public String toString() {
        return "Judul: " + judul_2009 +
            " | Penyanyi: " + penyanyi_2009 +
            " | Durasi: " + durasi_2009 + " detik";
    }
}
```

4.1.2 Sorting_2511532009

```
package pekan8_2511532009;
import java.util.Scanner;

public class Sorting_2511532009 {

    private Lagu_2511532009[] dataLagu_2009;
    private int jumlahLagu_2009;

    public Sorting_2511532009() {
        dataLagu_2009 = new Lagu_2511532009[20];
        jumlahLagu_2009 = 0;
    }
}
```

```

    }

    public void inputData_2009(Scanner scanner_2009) {
        while (true) {
            try {
                System.out.print("Masukkan jumlah lagu (min 7, maks 20): ");
                jumlahLagu_2009 = Integer.parseInt(scanner_2009.nextLine());

                if (jumlahLagu_2009 >= 7 && jumlahLagu_2009 <= 20) {
                    break;
                } else {
                    System.out.println("[Error] Jumlah lagu harus di antara 7 dan 20. Silakan coba lagi.");
                }
            } catch (NumberFormatException e) {
                System.out.println("[Error] Input tidak valid! Harap masukkan angka bulat.");
            }
        }

        System.out.println("\nMasukkan data lagu:");
        for (int i_2009 = 0; i_2009 < jumlahLagu_2009; i_2009++) {
            System.out.println("Lagu ke-" + (i_2009 + 1) + ":");

            String judul_2009;
            while (true) {
                System.out.print("Judul      : ");
                judul_2009 = scanner_2009.nextLine().trim();
                if (!judul_2009.isEmpty()) break;
                System.out.println("[Error] Judul lagu tidak boleh kosong!");
            }

            String penyanyi_2009;
            while (true) {
                System.out.print("Penyanyi : ");
                penyanyi_2009 = scanner_2009.nextLine().trim();
                if (!penyanyi_2009.isEmpty()) break;
                System.out.println("[Error] Nama penyanyi tidak boleh kosong!");
            }

            int durasi_2009;
            while (true) {
                try {
                    System.out.print("Durasi (detik) : ");
                    durasi_2009 = Integer.parseInt(scanner_2009.nextLine());
                    if (durasi_2009 > 0) break;
                    System.out.println("[Error] Durasi harus lebih besar dari 0 detik!");
                } catch (NumberFormatException e) {
                    System.out.println("[Error] Input tidak valid! Harap masukkan angka bulat untuk durasi.");
                }
            }

            dataLagu_2009[i_2009] = new Lagu_2511532009(judul_2009, penyanyi_2009, durasi_2009);
        }
    }
}

```

```

public void shellSort_2009() {
    int n_2009 = jumlahLagu_2009;

    for (int gap_2009 = n_2009 / 2; gap_2009 > 0; gap_2009 /= 2) {
        for (int i_2009 = gap_2009; i_2009 < n_2009; i_2009++) {
            Lagu_2511532009 temp_2009 = dataLagu_2009[i_2009];
            int j_2009;

            for (j_2009 = i_2009; j_2009 >= gap_2009 &&
                dataLagu_2009[j_2009 - gap_2009].getJudul_2009()
                    .compareToIgnoreCase(temp_2009.getJudul_2009()) > 0;
                j_2009 -= gap_2009) {
                dataLagu_2009[j_2009] = dataLagu_2009[j_2009 - gap_2009];
            }

            dataLagu_2009[j_2009] = temp_2009;
        }
    }
}

public void tampilData_2009() {
    System.out.println();
    System.out.println("Data Sebelum Sorting:");
    for (int i_2009 = 0; i_2009 < jumlahLagu_2009; i_2009++) {
        System.out.println("\n" + dataLagu_2009[i_2009].getJudul_2009()
            + " - " + dataLagu_2009[i_2009].getDurasi_2009() + " detik");
    }

    shellSort_2009();

    System.out.println();
    System.out.println("Data Setelah Shell Sort (Judul Asc):");
    for (int i_2009 = 0; i_2009 < jumlahLagu_2009; i_2009++) {
        System.out.println((i_2009 + 1) + ". " +
            dataLagu_2009[i_2009].getJudul_2009()
                + " - " + dataLagu_2009[i_2009].getDurasi_2009() + " detik");
    }
}

public static void main(String[] args) {
    Scanner scanner_2009 = new Scanner(System.in);

    System.out.println("=== Sorting Playlist NIM : 2511532009 ===");
    System.out.println("Algoritma : Shell Sort (Ascending Judul)");

    Sorting_2511532009 playlist_2009 = new Sorting_2511532009();
    playlist_2009.inputData_2009(scanner_2009);
    playlist_2009.tampilData_2009();

    scanner_2009.close();
}
}

```

4.2 Output yang dihasilkan

```
<terminated> Sorting_2511532009 [Java Application] C:\
kartu - 1234 detik

afal - 12 detik

bunnadd - 123 detik

padamu - 431 detik

selew - 980 detik

ruat - 140 detik

Data Setelah Shell Sort (Judul Asc):
1. afal - 12 detik
2. bunnadd - 123 detik
3. kartu - 1234 detik
4. padamu - 431 detik
5. ruat - 140 detik
6. selew - 980 detik
7. tarot - 160 detik
8. uhh - 124 detik
```

DAFTAR PUSTAKA

- [1] Modul Praktikum Struktur Data Pekan 8
- [2] Data Structures and Algorithms in Java